

ICTP DP Linux Basic Course - Fortran

ESP Students - First Semester

Graziano Giuliani
ggiulian@ictp.it

The Abdus Salam International Centre for Theoretical Physics

ICTP Diploma Program
September 26, 2025

Course Outline ¹

Daily program

- **UNIX/Linux**
- **Programming on Linux**
- **Text file manipulation**
- **Basic BASH and Python**
- **Basic Fortran**
 - 1 Program
 - 2 Modules and Build System
 - 3 Libraries

Slides:

<http://tinyurl.com/2jsvfbd6>

or the $\text{\LaTeX}$ source on GitHub:

<https://github.com/graziano-giuliani/LinuxBasics>

¹Course created in 2019 with Adriano Angelone, now LPTMC-FR

Fortran Program

Text I/O

Simple I/O Program

```
program simple_io
!
! This program reads in and prints out a name
!
implicit none
character (len=20) :: first_name

print *, ' type in your first name.'
print *, ' up to 20 characters'
read *, first_name
print *, ' you enter this name: ', first_name
end program
```

Programming Tips

Guidelines

- 1 Use comments to clarify what is the purpose of program units
- 2 Use meaningful variable names
- 3 INDENTATION is not mandatory, BUT... DO INDENT!!
- 4 Old FORTRAN did allow for implicit definitions: don't use it.
- 5 Decide on your own personal conventions and STICK to them
- 6 Use appropriate types for variables
- 7 Use as much parallelism as possible from the start

Formula Translator

Why Fortran

1 Fortran Intrinsic

- <https://gcc.gnu.org/onlinedocs/gfortran/Intrinsic-Procedures.html>

2 Arrays

- Custom bounds
- Slicing and striding
- Dynamic allocation

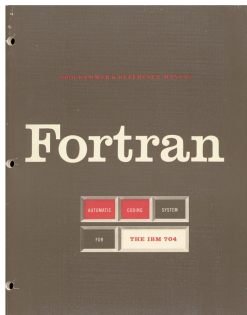
3 Array operations element by element using implicit loops

4 Vectorized scalar function using *elemental* keyword

5 Object orientation and procedure overloading

Fortran Language

Programming like in the 1950



- Fortran programs are written as source text files:

```
program count
  integer :: i
  do i = 1, 10
    print *, i
  end do
end program count
```

- A **compiler program** parses source files and create binary object files:

```
gfortran -o myprog myprog.f90
```

- Objects are linked with other objects or libraries to create executables:

```
00000000 457f 464c 0102 0001 0000 0000 0000 0000
00000100 0003 003e 0001 0000 06f0 0000 0000 0000
00000200 0040 0000 0000 0000 1a78 0000 0000 0000
00000300 0000 0000 0040 0038 0009 0040 001d 001c
00000400 0006 0000 0004 0000 0040 0000 0000 0000
```

Exercise

Write and compile Fortran 'Hello World'

- Create a text file `hello.f90` with the below content:

```
program hello
  implicit none
  integer , parameter :: stdout = 6
  character(len=*), parameter :: message = 'Helo World'
  character(len=1), parameter :: mark = '!'
  write(stdout,'(a,a)') message, mark
end program hello
```

- Compile the program into an executable file:

```
gfortran -o hello hello.f90
```

- Execute the executable file created:

```
./hello
```

Have you noticed?

No error is allowed in the code: Fortran Language has very strict syntax.
But you can misspell the message!

Why we had to run the program with `./hello`?

Compiler flags

Optional arguments for Fortran programming

The compiler is a program and accepts command line arguments. Above we have used the `-o` option to specify the name of the output.

Useful gfortran options

- `-o program` output program name (default a.out)
- `-Ofast` Optimize program for fast execution time
- `-O0` Remove all optimization (for debugging)
- `-g` include debugging information
- `-Wall` Enables commonly used warning options pertaining to usage recommend avoiding and that are easy to avoid
- `-pedantic` check program for Fortran standard conformance
- `-fcheck=all` perform all available run-time checks
- `-fbacktrace` print the whole trace of the error

Exercise

Debug a Fortran program

- Create a text file `error.f90` with the below content:

```
program error
  implicit none
  integer, parameter :: im = 2
  integer, dimension(im,im) :: matrix
  integer :: j
  do j = 1 , 2*im
    do i = 1 , 2*im
      matrix(i,j) = i*j
      print*, matrix(i,j)
    end do
  end do
end program error
```

- Compile and correct the error
- Run the program and check the runtime error
- Compile in debug: `-O0 -g -Wall -pedantic -fcheck=all -fbacktrace`
- Run the program, check the error and correct the program
- Recompile an optimized executable

Program Example

A numerical Exercise

• Assignment from Numerical

Write a program that solves the differential equation
 $dx/dt = -x$ using

- 1) Euler method
- 2) midpoint methods.

Create separate subroutines for the two methods.

Solve with initial conditions $x(0.0) = 1.0$ and $dt=0.1$,
 $N=100$ and store the results in arrays `tlist` and `xlist`.

The program should create a file for each method with
the result in two columns: `t` and `x(t)`.

Plot the result `x` versus `t` with your favorite plotting
program.

Submit the graphs and your code.